

DDA5001 Machine Learning

Linear classification I: The Perceptron
& Linear Regression: Least Squares
(Week 2, Part 1)

Junchi Yang

School of Data Science
The Chinese University of Hong Kong, Shenzhen

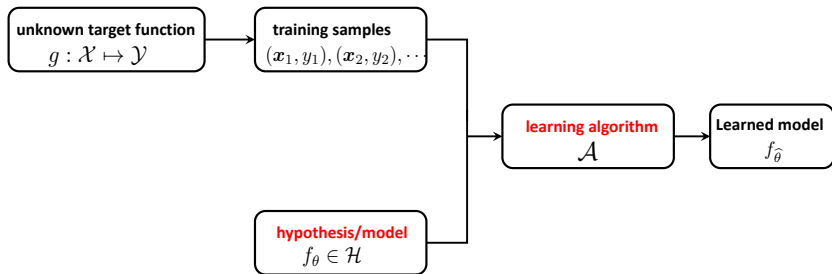


Recap: Supervised Learning Components

Components:

- ▶ Target function $g : \mathcal{X} \rightarrow \mathcal{Y}$ (underlying credit approval model)
- ▶ Training dataset: $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ (historical records)
- ▶ Hypothesis space $\mathcal{H}$ (learning scope to approximate g)
- ▶ Hypothesis/model: f_{θ} (model to be determined)
- ▶ Optimization algorithm: $\mathcal{A}$ (learning the model from data)

Recap: Supervised Learning: High-level View



Hopefully,

$$f_{\hat{\theta}} \approx g$$

Inference

Predict/decision: When a new sample data (test data) x comes, the label is predicted as:

$$y \leftarrow f_{\hat{\theta}}(x).$$

$\rightsquigarrow$ We now discuss linear models for classification and regression.

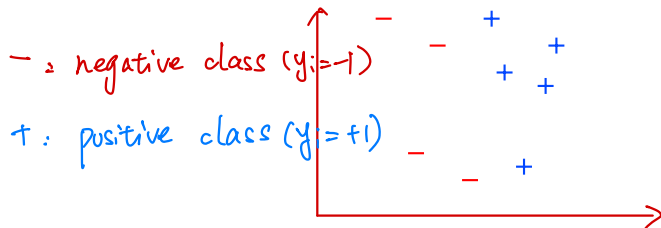
Agenda

Linear Classification I — The Perceptron

Linear Regression — Least Squares

Classification

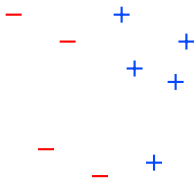
Consider the following illustration of the credit approval example:



- In the above figure, we can regard each point as one applicant's data (sample). This forms the training data pairs $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ with $y_i = \pm 1$ (approve or deny).

Classification

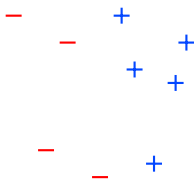
Consider the following illustration of the credit approval example:



- ▶ In the above figure, we can regard each point as one applicant's data (sample). This forms the training data pairs $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ with $y_i = \pm 1$ (approve or deny).
- ▶ **Classification** is to learn a model $f_{\theta} : \mathbb{R}^d \rightarrow \{-1, +1\}$ based on the training samples to correctly classify the training samples (to approximate the underlying unknown g). This is called **binary classification** since we only have two classes.

Classification

Consider the following illustration of the credit approval example:

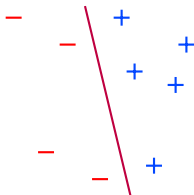


- ▶ In the above figure, we can regard each point as one applicant's data (sample). This forms the training data pairs $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ with $y_i = \pm 1$ (approve or deny).
- ▶ **Classification** is to learn a model $f_{\theta} : \mathbb{R}^d \rightarrow \{-1, +1\}$ based on the training samples to correctly classify the training samples (to approximate the underlying unknown g). This is called **binary classification** since we only have two classes.

What type of model f_{θ} can we choose?

Linear Classification

Consider the following illustration of the credit approval example:



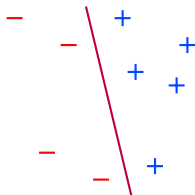
Linear classification: Find a linear model based on the training sample to correctly classify the training samples. Namely, f_{θ} takes the linear form

$$f_{\theta, b}(x) = \theta^{\top} x + b.$$

Correspondingly, the hypothesis space is given by

$$\mathcal{H} := \{f_{\theta, b}(x) = \theta^{\top} x + b : \theta \in \mathbb{R}^n, b \in \mathbb{R}\},$$

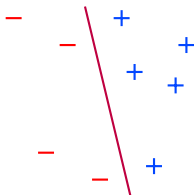
Linear Classification



The linear model is learned to separate the dataset:

$$\begin{cases} f_{\theta, \mathbf{b}}(\mathbf{x}_i) > 0 & \text{if } y_i = +1, \\ f_{\theta, \mathbf{b}}(\mathbf{x}_i) < 0 & \text{if } y_i = -1, \end{cases} \quad \forall i = 1, \dots, n.$$

Linear Classification



The linear model is learned to separate the dataset:

$$\begin{cases} f_{\theta, \mathbf{b}}(\mathbf{x}_i) > 0 & \text{if } y_i = +1, \\ f_{\theta, \mathbf{b}}(\mathbf{x}_i) < 0 & \text{if } y_i = -1, \end{cases} \quad \forall i = 1, \dots, n.$$

The classification result can be represented using the “sign” function, i.e.,

$$y_i = \text{sign}(f_{\theta, \mathbf{b}}(\mathbf{x}_i)),$$

where

$$\text{sign}(z) = \begin{cases} +1, & \text{if } z > 0, \\ -1, & \text{if } z < 0, \\ [-1, +1], & \text{if } z = 0. \end{cases}$$

Absorbing the Bias Term

- ▶ The term “ b ” in the linear model $f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x} + b$ is called **bias** term.
- ▶ Let $\theta_0 = b$ and $x_0 = 1$, and now, we define

$$\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{d+1}$$

$$\tilde{\boldsymbol{\theta}} = (\theta_0, \boldsymbol{\theta}) \in \mathbb{R}^{d+1}$$

It then follows that

$$\tilde{\boldsymbol{\theta}}^\top \tilde{\mathbf{x}} = \boldsymbol{\theta}^\top \mathbf{x} + \theta_0 x_0 = \boldsymbol{\theta}^\top \mathbf{x} + b.$$

Absorbing the Bias Term

- ▶ The term “ b ” in the linear model $f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^{\top} \mathbf{x} + b$ is called **bias** term.
- ▶ Let $\theta_0 = b$ and $x_0 = 1$, and now, we define

$$\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{d+1}$$

$$\tilde{\boldsymbol{\theta}} = (\theta_0, \boldsymbol{\theta}) \in \mathbb{R}^{d+1}$$

It then follows that

$$\tilde{\boldsymbol{\theta}}^{\top} \tilde{\mathbf{x}} = \boldsymbol{\theta}^{\top} \mathbf{x} + \theta_0 x_0 = \boldsymbol{\theta}^{\top} \mathbf{x} + b.$$

In the sequel, we will use $\boldsymbol{\theta}$ and $\mathbf{x}$ to represent $\tilde{\boldsymbol{\theta}}$ and $\tilde{\mathbf{x}}$ to ease notation, respectively. Then, we can compactly write the linear model as

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^{\top} \mathbf{x} \quad \text{and} \quad y = \text{sign}(f_{\boldsymbol{\theta}}(\mathbf{x})).$$

In this way, the bias term is absorbed and the linear model representation is more compact.

Absorbing the Bias Term

- ▶ The term “ b ” in the linear model $f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^{\top} \mathbf{x} + b$ is called **bias** term.
- ▶ Let $\theta_0 = b$ and $x_0 = 1$, and now, we define

$$\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{d+1}$$

$$\tilde{\boldsymbol{\theta}} = (\theta_0, \boldsymbol{\theta}) \in \mathbb{R}^{d+1}$$

It then follows that

$$\tilde{\boldsymbol{\theta}}^{\top} \tilde{\mathbf{x}} = \boldsymbol{\theta}^{\top} \mathbf{x} + \theta_0 x_0 = \boldsymbol{\theta}^{\top} \mathbf{x} + b.$$

In the sequel, we will use $\boldsymbol{\theta}$ and $\mathbf{x}$ to represent $\tilde{\boldsymbol{\theta}}$ and $\tilde{\mathbf{x}}$ to ease notation, respectively. Then, we can compactly write the linear model as

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^{\top} \mathbf{x} \quad \text{and} \quad y = \text{sign}(f_{\boldsymbol{\theta}}(\mathbf{x})).$$

In this way, the bias term is absorbed and the linear model representation is more compact.

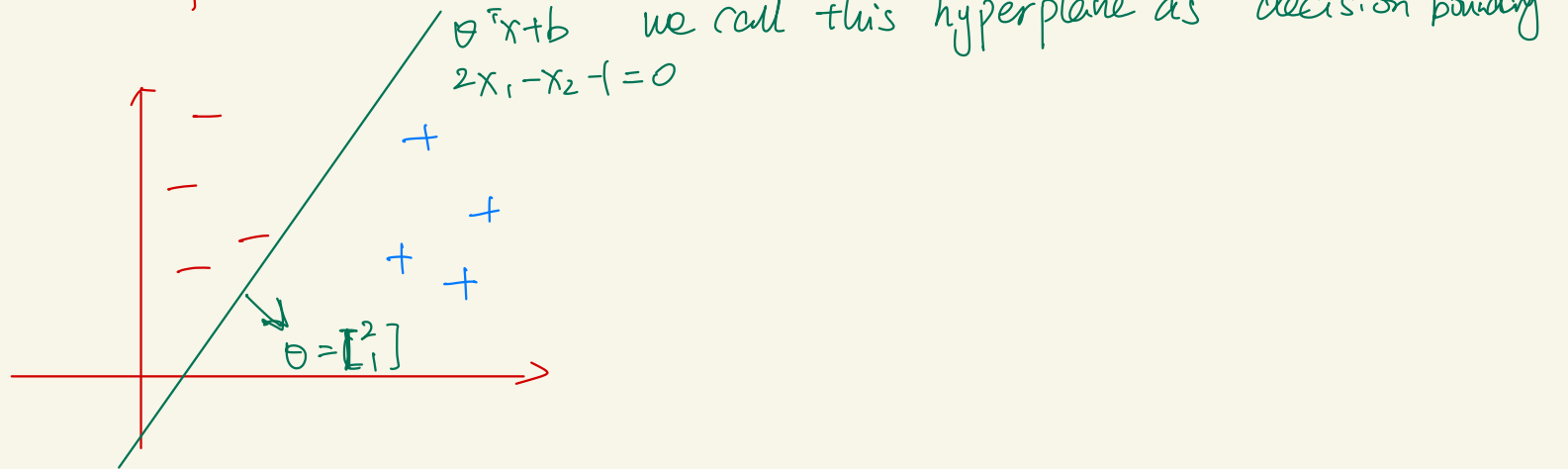
↪ After choosing the model, we need an algorithm to find the classifier.

Do not just ignore bias term: e.g. $x_1 = 1$ (negative)

$x_2 = 2$ (positive)

any θx cannot separate them because θx_1 and θ_2 always have the same sign. But $f_{\theta,b}(x) = x - 1.5$.

Consider $f_{\theta,b}(x) = \theta^T x + b$



For $\theta \neq 0$, the signed Euclidean distance
to the hyperplane

$$\frac{\theta^T x + b}{\|\theta\|_2}$$



positive on the side θ points to
negative on the other side.

θ = decide which direction count as positive.

b : where the dividing line (hyperplane) sit.

Linear Classification Algorithm: The Perceptron

- ▶ The **perceptron** was invented in 1943 by McCulloch and Pitts.
- ▶ It is a simple linear classification algorithm used to find a separation line based on the training dataset.
- ▶ The perceptron was intended to be a machine, rather than a program, and while its first implementation was in software for the IBM 704.
- ▶ Later in 1957, it was implemented in hardware-level machine and the machine is called "Mark I perceptron", designed for image recognition.

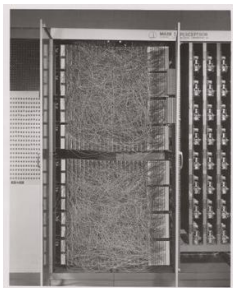


Figure: Mark I Perceptron machine.

The Perceptron: Algorithm Procedure

- ▶ Given the training dataset $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$, with $y_i = \pm 1$.
- ▶ Linear classification model

$$f_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x}, \quad y = \text{sign}(f_{\theta}(\mathbf{x})).$$

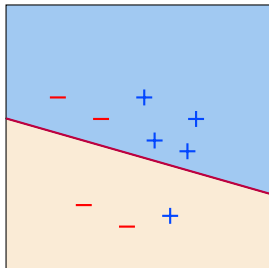
- ▶ **Intuition:** θ should be in the same direction as the points $\mathbf{x}_i$ in the $y = +1$ class, and in the opposite direction as those in the $y = -1$ class
- ▶ **Optimization:** Pick a **misclassified** data $\mathbf{x}_i$ (any misclassified one)

$$\underbrace{\text{sign}(f_{\theta}(\mathbf{x}_i))}_{=\text{sign}(\theta^{\top} \mathbf{x}_i)} \neq y_i.$$

- ▶ Update rule

$$\theta = \theta + y_i \mathbf{x}_i.$$

for some i misclassified



$y_i \theta^T x_i > 0 \Rightarrow i$ is correctly classified

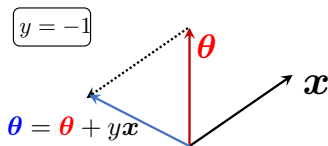
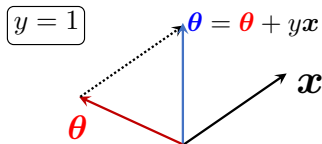
$y_i \theta^T x_i < 0 \Rightarrow$ misclassified.

The Perceptron: Interpretation

► Update rule

$$\theta = \theta + y_i x_i$$

- This update rule pushes the line to correctly classify the picked data. **Iteratively** repeat this procedure, until all training samples are correctly classified.



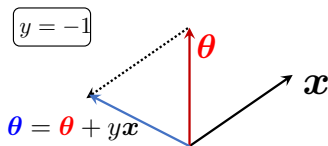
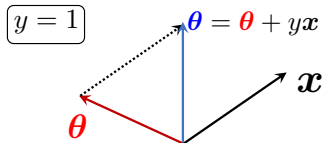
Intuition: when $y=1$, we want $\theta^T x > 0$, i.e., θ and x are broadly the same direction.
when $y=-1$, we want $\theta^T x < 0$, i.e., θ and x are broadly opposite direction.

The Perceptron: Interpretation

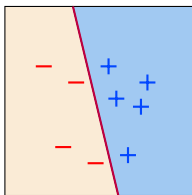
► Update rule

$$\theta = \theta + y_i x_i$$

- This update rule pushes the line to correctly classify the picked data. **Iteratively** repeat this procedure, until all training samples are correctly classified.



Learned model:



Perceptron Example: only 2 samples : A : (x=1, y=-1)
B : (x=2, y=+1)

$$A: \tilde{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad y = -1$$

$$B: \tilde{x}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad y = +1$$

start from $\theta = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Iteration 1: A is misclassified since $y_1 \theta^T \tilde{x}_1 = -3$.

$$\theta \leftarrow \theta + y_1 \tilde{x}_1 \quad \theta = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + (-1) \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Iter 2: A is misclassified since $y_1 \theta^T \tilde{x}_1 = -1$

$$\theta \leftarrow \theta + y_1 \tilde{x}_1 \quad \theta = \begin{bmatrix} 0 \\ 1 \end{bmatrix} + (-1) \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

Iteration 3: B is misclassified

$$\theta \leftarrow \theta + y_2 \tilde{x}_2 \quad \theta = \begin{bmatrix} -1 \\ 0 \end{bmatrix} + 1 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

Example

Which statement is true?

- ~~(A)~~ The perceptron algorithm is used for unsupervised classification.
- ~~(B)~~ The perceptron algorithm can find a classifier (separation line) in one update step.
- ~~(C)~~ The perceptron algorithm will stop at the same separation line for every possible runs.
- (D) The perceptron is tailored for binary classification.

Example

Which statement is true?

- (A) The perceptron algorithm is used for unsupervised classification.
- (B) The perceptron algorithm can find a classifier (separation line) in one update step.
- (C) The perceptron algorithm will stop at the same separation line for every possible runs.
- (D) The perceptron is tailored for binary classification.

Answer: (D). For (A), we have label, hence it is supervised. Actually, classification always has label and is always supervised. For (B), it depends on the data. It may take many iterative steps to converge. For (C), it may stop at different separation lines, depending on which misclassified data to choose.

Agenda

Linear Classification I — The Perceptron

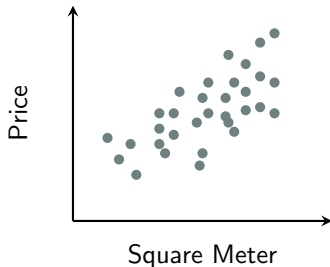
Linear Regression — Least Squares

Regression *Also Supervised learning.*

Data: Given past sales data $\{(x_i, y_i)\}_{i=1}^n$, where

$x_i \in \mathbb{R}$ represents square meter

y_i represents apartment price



Regression is to learn a model f_θ that captures relationship between x_i and y_i . Given a new apartment with square meter x , predict $y = f_\theta(x)$.

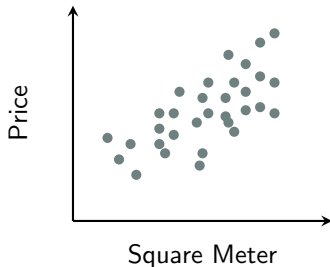
Regression means that y is real-valued (continuous).

Regression

Data: Given past sales data $\{(x_i, y_i)\}_{i=1}^n$, where

$x_i \in \mathbb{R}$ represents square meter

y_i represents apartment price



Regression is to learn a model f_θ that captures relationship between x_i and y_i . Given a new apartment with square meter x , predict $y = f_\theta(x)$.

Regression means that y is **real-valued (continuous)**.

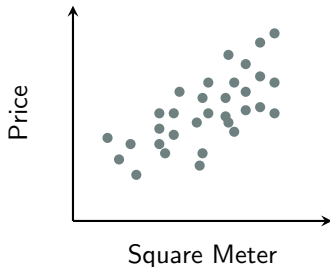
What is the difference between regression and classification?

Regression

Data: Given past sales data $\{(x_i, y_i)\}_{i=1}^n$, where

$x_i \in \mathbb{R}$ represents square meter

y_i represents apartment price



Regression is to learn a model f_θ that captures relationship between x_i and y_i . Given a new apartment with square meter x , predict $y = f_\theta(x)$.

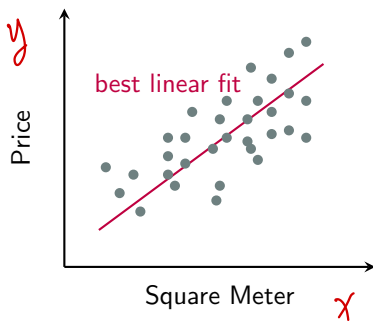
Regression means that **y is real-valued (continuous)**.

What is the difference between regression and classification? In regression, y is continuous, while in classification it is categorical.

Linear Regression

- ▶ If we choose the hypothesis space to be linear model, then we have **linear regression**. That is, we assume that the apartment price is a **linear** function of square meter.
- ▶ For the previous simple example, the linear model can be expressed as

$$y_i \approx f_{\theta}(x_i) = \theta x_i.$$



Multi-dimensional Linear Regression

Data: Given past sales data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where

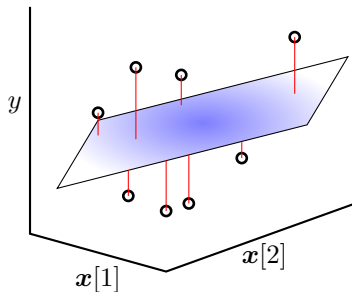
$\mathbf{x}_i \in \mathbb{R}^d$ represents square meter, metro, schools...

y_i represents price

Linear model:

$$y_i \approx f_{\boldsymbol{\theta}}(\mathbf{x}_i) = \boldsymbol{\theta}^{\top} \mathbf{x}_i,$$

where $\boldsymbol{\theta} \in \mathbb{R}^d$.



How to find the best linear fit? Least squares.

Least Squares

The learning problem can be formulated as:

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \ell(\underline{f_{\theta}(\mathbf{x}_i)}, y_i)$$

$\theta^\top \mathbf{x}_i$: prediction for i th sample

Since our goal is to enforce $y_i \approx f_{\theta}(\mathbf{x}_i)$ and the label y is continuous real-valued. Thus, we can choose the most common ℓ_2 -loss function:

$$\ell(a, b) = (a - b)^2.$$

The choice of ℓ_2 -loss results in the famous least squares (LS) formulation:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n (\theta^\top \mathbf{x}_i - y_i)^2$$

average over
all data

prediction

true label

Least Squares

The learning problem can be formulated as:

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

Since our goal is to enforce $y_i \approx f_{\theta}(\mathbf{x}_i)$ and the label y is continuous real-valued. Thus, we can choose the most common ℓ_2 -loss function:

$$\ell(a, b) = (a - b)^2.$$

The choice of ℓ_2 -loss results in the famous **least squares (LS)** formulation:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n (\theta^{\top} \mathbf{x}_i - y_i)^2$$

↪ We can also select absolute value loss, leading to robust linear regression formulation (Homework 1).

Least Squares in Matrix Notation

Matrix representation of data and label:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n$$

each row represent one sample
d: dim of feature
n: number of samples

Fact:

$$\begin{bmatrix} \boldsymbol{\theta}^\top \mathbf{x}_1 - y_1 \\ \vdots \\ \boldsymbol{\theta}^\top \mathbf{x}_n - y_n \end{bmatrix} = \mathbf{X}\boldsymbol{\theta} - \mathbf{y} \in \mathbb{R}^n$$

Least squares:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n (\boldsymbol{\theta}^\top \mathbf{x}_i - y_i)^2 \iff \boxed{\min_{\boldsymbol{\theta}} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2}$$

Solution of Least Squares

Solution of Least Squares

Let (we can ignore the $\frac{1}{n}$ without loss of generality)

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Then

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \\ &= \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}\end{aligned}$$

Solution of Least Squares

Let (we can ignore the $\frac{1}{n}$ without loss of generality)

$$\mathcal{L}(\theta) = \|X\theta - y\|_2^2$$

Then

$$\begin{aligned}\mathcal{L}(\theta) &= (X\theta - y)^\top (X\theta - y) \\ &= \theta^\top \underbrace{X^\top X \theta}_{\text{symmetric}} - 2\theta^\top X^\top y + y^\top y\end{aligned}$$

Facts:

- ▶ If $h(\theta) = c^\top \theta$, then $\nabla h(\theta) = c$.
- ▶ If $h(\theta) = \theta^\top M \theta$ (M is symmetric), then: $\nabla h(\theta) = 2M\theta$.

$$\nabla \mathcal{L}(\theta) = 2X^\top X \theta - 2X^\top y$$

Solution of Least Squares

Let (we can ignore the $\frac{1}{n}$ without loss of generality)

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Then

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \\ &= \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}\end{aligned}$$

Facts:

- ▶ If $h(\boldsymbol{\theta}) = \mathbf{c}^\top \boldsymbol{\theta}$, then $\nabla h(\boldsymbol{\theta}) = \mathbf{c}$.
- ▶ If $h(\boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{M} \boldsymbol{\theta}$ ($\mathbf{M}$ is symmetric), then: $\nabla h(\boldsymbol{\theta}) = 2\mathbf{M}\boldsymbol{\theta}$.

Take the gradient

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = 2\mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Solution of Least Squares

Let (we can ignore the $\frac{1}{n}$ without loss of generality)

$$\mathcal{L}(\theta) = \|X\theta - y\|_2^2$$

Then

$$\begin{aligned}\mathcal{L}(\theta) &= (X\theta - y)^\top (X\theta - y) \\ &= \theta^\top X^\top X\theta - 2\theta^\top X^\top y + y^\top y\end{aligned}$$

Facts:

- ▶ If $h(\theta) = c^\top \theta$, then $\nabla h(\theta) = c$.
- ▶ If $h(\theta) = \theta^\top M\theta$ (M is symmetric), then: $\nabla h(\theta) = 2M\theta$.

Take the gradient

$$\nabla_{\theta} \mathcal{L}(\theta) = 2X^\top (X\theta - y)$$

Setting the gradient to zero and due to **convexity** of $\mathcal{L}$, the optimal solution $\hat{\theta}$ satisfies

$$\boxed{X^\top X \hat{\theta} = X^\top y} \quad (\text{normal equation})$$

How to solve for $\hat{\theta}$?

Verify any $\hat{\theta}$ satisfying $X^T X \hat{\theta} = X^T y$ is optimal.

Method 1: For unconstrained convex function, $\nabla L(\hat{\theta}) = 0$ means $\hat{\theta}$ is global optimal solution.

Hessian of L is $2X^T X$, which is positive semi-definite. $\Rightarrow L$ is convex

Method 2: $L(\hat{\theta} + h) = \|X\hat{\theta} - y + Xh\|^2$

$$= L(\hat{\theta}) + \underbrace{2h^T X^T (X\hat{\theta} - y)}_{=0} + \|Xh\|^2$$

$$= L(\hat{\theta}) + \|Xh\|^2 \geq L(\hat{\theta})$$

Solution of Least Squares: Case I

Case I: $\mathbf{X} \in \mathbb{R}^{n \times d}$ has full column rank (its rank is d)

- ▶ $\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$ will be full rank / invertible (Homework 1).
- ▶ We have

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad \text{unique solution}$$

Pseudo-inverse of matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ when it has full column rank is defined as

$$\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \in \mathbb{R}^{d \times n}.$$

- ▶ We have $\mathbf{X}^\dagger \mathbf{X} = \mathbf{I}$, but not necessarily $\mathbf{X} \mathbf{X}^\dagger = \mathbf{I}$.

Hence, we can also write

$$\hat{\boldsymbol{\theta}} = \mathbf{X}^\dagger \mathbf{y}.$$

Solution of Least Squares: Case I

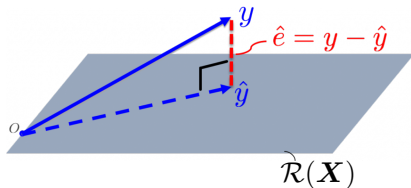
Geometric interpretation:

- Let

$$\mathcal{P}_X = \mathbf{X}\mathbf{X}^\dagger.$$

- $\mathcal{P}_X$ is the **projection matrix** onto the range space $\mathcal{R}(\mathbf{X})$: the space of all possible predictions $\mathbf{X}\boldsymbol{\theta}$ our linear model can make.
- What does the LS solution mean?

$$\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{y} - \mathbf{X}\mathbf{X}^\dagger \mathbf{y} = \mathbf{y} - \mathcal{P}_X(\mathbf{y}) := \mathbf{y} - \hat{\mathbf{y}}.$$



Conclusion: The LS solution is such that $\hat{\mathbf{y}} := \mathbf{X}\hat{\boldsymbol{\theta}} = \mathcal{P}_X(\mathbf{y})$ is the **orthogonal projection** of $\mathbf{y}$ onto the range space $\mathcal{R}(\mathbf{X})$.

We define residual as $\hat{e} = y - X\hat{\theta} \in \mathbb{R}^n$

$\uparrow$ $\nwarrow$
true label prediction

if $\hat{\theta}$ satisfies normal equation,

$$\text{with } \hat{e} = y - X\hat{\theta}, \quad X^T \hat{e} = X^T y - X^T X \hat{\theta} = 0$$

$\Rightarrow \hat{e}$ is perpendicular to column space of X .

Claim: $\sum_{i=1}^n \hat{e}_i = 0$ if X includes the all 1's column
(the column absorbs bias term) $X = \begin{bmatrix} 1 & \dots & \end{bmatrix}$

because $X^T \hat{e} = 0 \Rightarrow [1, \dots, 1] \hat{e} = 0$, i.e. $\sum \hat{e}_i = 0$

Solution of Least Squares: Case II

Case II: $\mathbf{X} \in \mathbb{R}^{n \times d}$ does not have full column rank (its rank is $< d$).

- One typical case is, $n < d$.

Solution of Least Squares: Case II

Case II: $\mathbf{X} \in \mathbb{R}^{n \times d}$ does not have full column rank (its rank is $< d$).

- ▶ One typical case is, $n < d$. **It means overfitting.**
- ▶ We do not have a unique solution. Instead, we have **infinitely many** solutions.

We can represent all the solutions using singular value decomposition (SVD), and it will be discussed in Homework 1.

Solution of Least Squares: Case II

Case II: $\mathbf{X} \in \mathbb{R}^{n \times d}$ does not have full column rank (its rank is $< d$).

- ▶ One typical case is, $n < d$. **It means overfitting.**
- ▶ We do not have a unique solution. Instead, we have **infinitely many** solutions.

We can represent all the solutions using singular value decomposition (SVD), and it will be discussed in Homework 1.

How to obtain a unique and meaningful solution in this case?

↪ Regularization, which we will study in details later in the overfitting chapter (simple derivation will be discussed in Homework 1).

↪ Next lecture: Maximum likelihood estimation and feasibility of learning.